

הרצת תהליכים ב-JAVA

מערכות הפעלה

הרצת תהליכים ב-JAVA

- נבחן כיצד להריץ תהליכים דרך תוכנה שאנו כותבים (ספציפית ב-JAVA)
- נבחין בין שני מקרים:
 1. הרצה פשוטה של תוכנה אחרת
 2. הרצת תהליך

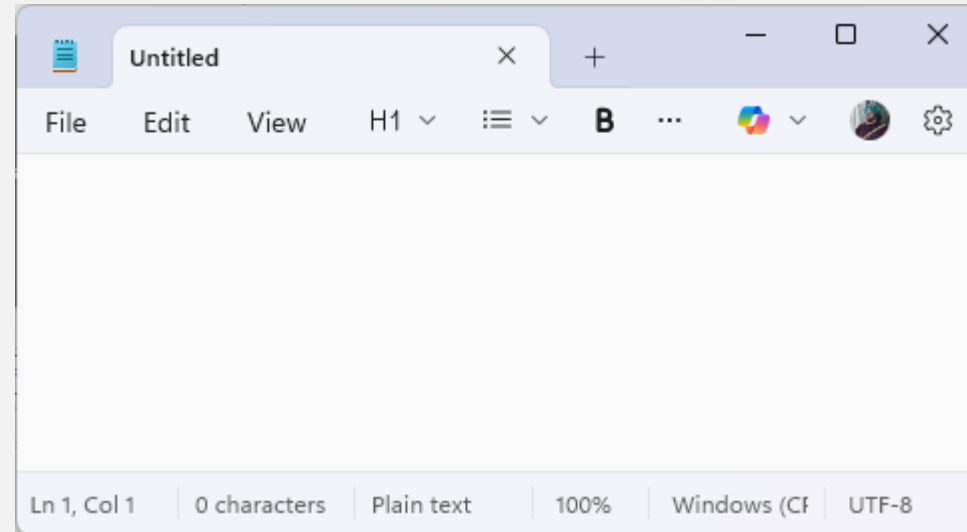
הרצה של תוכנה אחרת

- הטיפול המובנה Runtime מייצג אובייקט שמאפשר ל-JAVA לתקשר עם מערכת ההפעלה
- כדי לקבל אותו יש להפעיל את הפונקציה הסטטית `Runtime.getRuntime()`
- לאובייקט זה יש שיטה בשם `exec` המאפשרת הרצה של תוכנה
- השיטה מקבלת מערך של מחרוזות, כאשר ומריצה את התוכנה שנמצאת בכתובות השמורה בתא הראשון של המערך
- יש להפעיל שיטה זו עטופה ב-`try/catch`, כיוון שעלולות להיות שגיאות בתקשורת עם המערכת
- לדגומה, תוכנת JAVA הפותחת את `notepad`

```
String[] prog = {"notepad.exe"};
try {
    Runtime.getRuntime().exec(prog);
    System.out.println("Successfully opened notepad");
} catch (Exception e) {
    e.printStackTrace();
}
```

```
String[] prog = {"notepad.exe"};
try {
    Runtime.getRuntime().exec(prog);
    System.out.println("Successfully opened notepad");
} catch (Exception e) {
    e.printStackTrace();
}
```

Successfully opened notepad

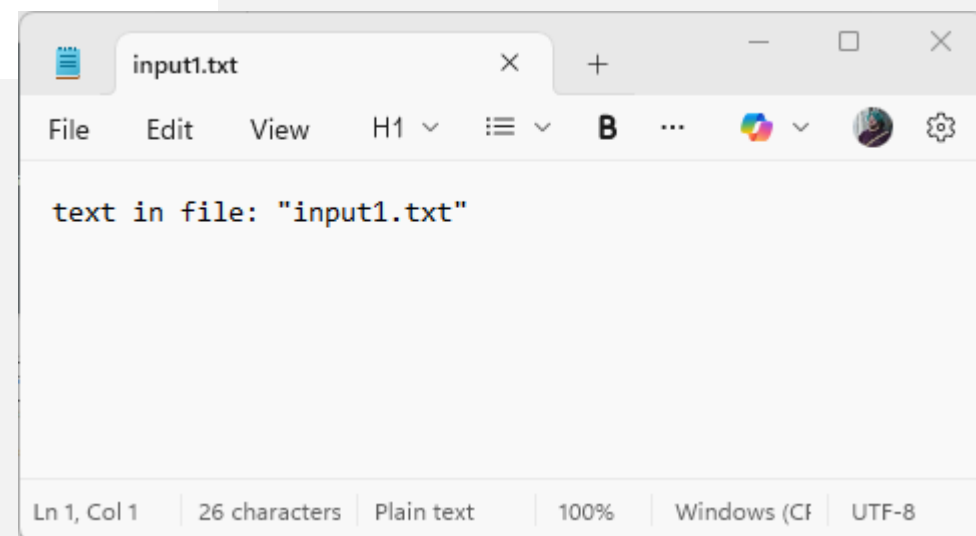


הרצה של תוכנה אחרת

- ניתן גם להוסיף ערכים נוספים במערך, שיהיו הארגומנטים שאיתם תופעל התוכנה
- לדוגמא, פתחית notepad עם קובץ מסויים

```
String[] prog = {"notepad.exe", "C:\\files\\input1.txt"};  
try {  
    Runtime.getRuntime().exec(prog);  
    System.out.println("Successfully opened notepad");  
} catch (Exception e) {  
    e.printStackTrace();  
}
```

Successfully opened notepad



הרצה של תוכנה אחרת

- שיטה זו מוגבלת ביכולות שלה
- "שגר ושכח", ביקשנו מהתוכנה לרוץ, ואלא אם כן היא לא הצליחה לרוץ, אנחנו לא מקבלים שום פידבק בחזרה
- בשלב הבא נראה שיטה להריץ תהליכים
- מעבר ליכולת להריץ פקודות מורכבות, גם נוכל להשתמש בשיטה זו כדי להריץ תוכנות ולהמשיך להיות איתן בקשר
- זה יאפשר לנו בין היתר
- לקרוא את הפלט של התוכנה שהרצנו
- לחכות שהתוכנה תסיים לרוץ לפני שנמשיך

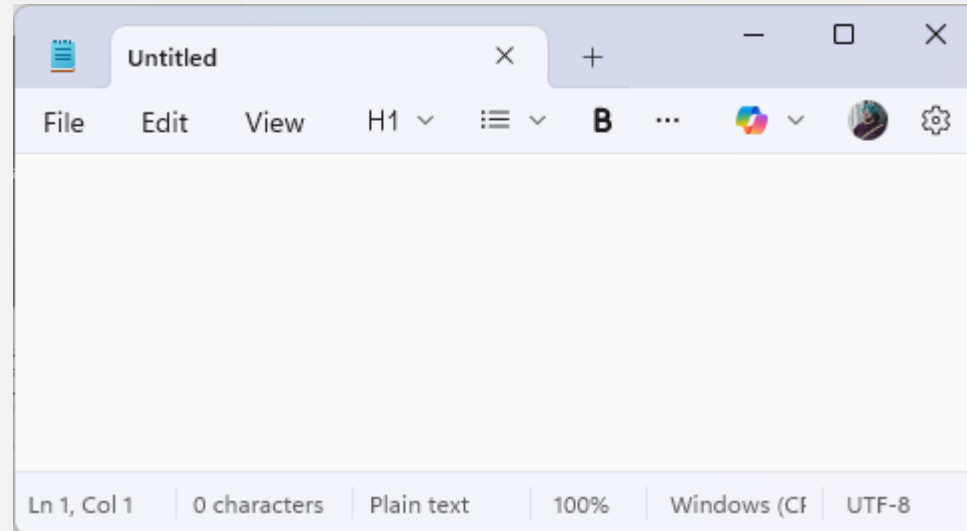
הרצת תהליך

- הטיפול המובנה Process מייצג תהליך שרץ במחשב
- הטיפול ProcessBuilder מייצג אובייקט שיכול לייצר לבנות תהליכים (אובייקטים מסוג Process) עבור תוכנה
- אפשר לחשוב עליו כעל "מפעל של תהליכים"
- כדי לייצר ProcessBuilder יש להפעיל בנאי עם כתובת התוכנה עבורה הוא יבנה תהליכים
- לProcessBuilder יש את השיטה start() אשר מריצה תהליך חדש עבור התוכנה ומייצר ערך Process שמקושר אליה
- לדוגמא (שוב) תוכנה שמריצה notepad

```
ProcessBuilder pb = new ProcessBuilder("notepad.exe");
try {
    Process process = pb.start();
    System.out.println("Successfully opened notepad");
    System.out.println("process id: "+process.pid());
} catch (Exception e) {
    e.printStackTrace();
}
```

```
ProcessBuilder pb = new ProcessBuilder("notepad.exe");
try {
    Process process = pb.start();
    System.out.println("Successfully opened notepad");
    System.out.println("process id: "+process.pid());
} catch (Exception e) {
    e.printStackTrace();
}
```

Successfully opened notepad
process id: 24072



הרצת תהליך

- pid (process id - מספר מזהה תהליך) ניתן לתהליך ע"י מערכת ההפעלה, ועשוי להיות שונה בכל

```
Successfully opened notepad  
procces id: 24072
```

```
Successfully opened notepad  
procces id: 8168
```

הרצה

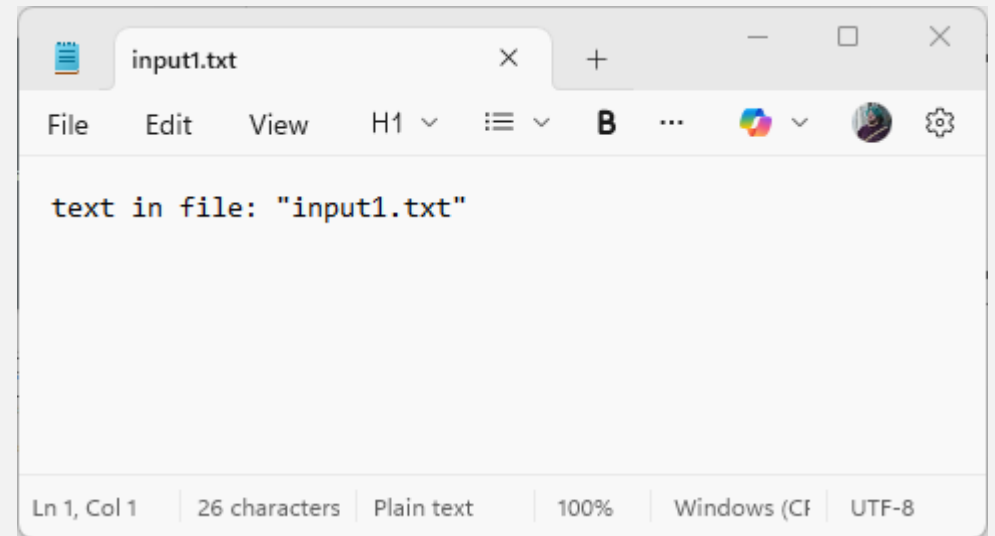
הרצת תהליך

- ניתן להוסיף ערכי String נוספים לבנאי של ProcessBuilder, שיהיו הארגומנטים של התוכנית שתרוץ
- לדוגמא (שוב), פתיחת notepad עם קובץ מסויים

```
ProcessBuilder pb = new ProcessBuilder("notepad.exe", "C:\\files\\input1.txt");
try {
    Process process = pb.start();
    System.out.println("Successfully opened notepad");
    System.out.println("process id: "+process.pid());
} catch (Exception e) {
    e.printStackTrace();
}
```

```
ProcessBuilder pb = new ProcessBuilder("notepad.exe", "C:\\files\\input1.txt");  
try {  
    Process process = pb.start();  
    System.out.println("Successfully opened notepad");  
    System.out.println("procces id: "+process.pid());  
} catch (Exception e) {  
    e.printStackTrace();  
}
```

Successfully opened notepad
procces id: 12220

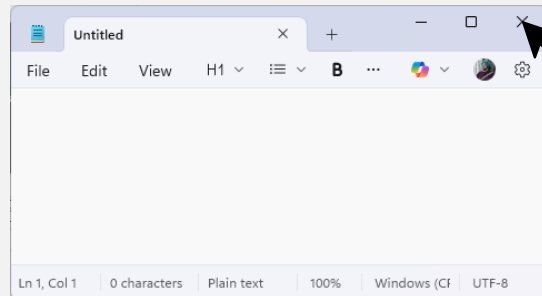


הרצת תהליך

- לאובייקט Process יש את השיטה waitFor() אשר גורמת לתוכנה לחכות עד שהתהליך יסיים את ריצתו

```
ProcessBuilder pb = new ProcessBuilder("notepad.exe");
try {
    Process process = pb.start();
    System.out.println("Successfully opened notepad");
    System.out.println("procces id: "+process.pid());
    process.waitFor();
    System.out.println("notepad ended");
} catch (Exception e) {
    e.printStackTrace();
}
```

Successfully opened notepad
procces id: 23888



Successfully opened notepad
procces id: 23888
notepad ended

הרצת תהליך

- כשתהליך מסיים את ריצתו הוא מחזיר מספר (exit code - קוד יציאה) אשר מספק מידע על האופן שבו הסתיים
- כל תוכנה יכולה להגדיר באיזה קוד היא יצאה, אך מקובל שקוד 0 מסמן "הסתיימה באופן תקין"
- השיטה `waitFor()` מחזירה את הקוד שאיתו סיים התהליך

```
ProcessBuilder pb = new ProcessBuilder("notepad.exe");
try {
    Process process = pb.start();
    System.out.println("Successfully opened notepad");
    System.out.println("process id: "+process.pid());
    int exitCode = process.waitFor();
    System.out.println("notepad ended with exit code: "+exitCode);
} catch (Exception e) {
    e.printStackTrace();
}
```

```
ProcessBuilder pb = new ProcessBuilder("notepad.exe");
try {
    Process process = pb.start();
    System.out.println("Successfully opened notepad");
    System.out.println("procces id: "+process.pid());
    int exitCode = process.waitFor();
    System.out.println("notepad ended with exit code: "+exitCode);
} catch (Exception e) {
    e.printStackTrace();
}
```

סגירה תקינה – דרך התוכנה

```
Successfully opened notepad
procces id: 24768
notepad ended with exit code: 0
```

סגירה מבחוץ – למשל הפסקת

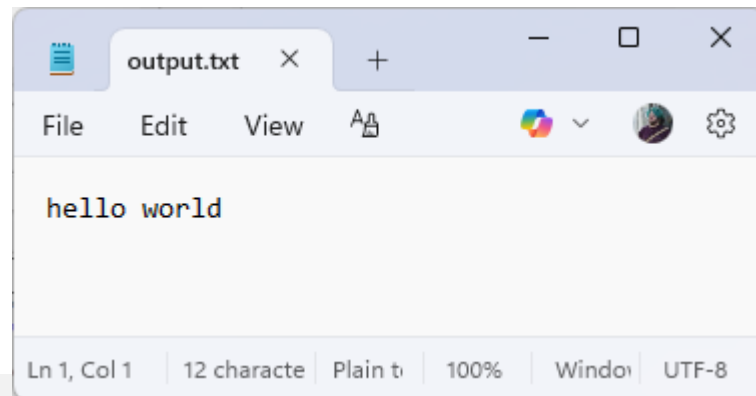
התהליך דרך taskManager

```
Successfully opened notepad
procces id: 18180
notepad ended with exit code: 1
```

הרצת תהליך

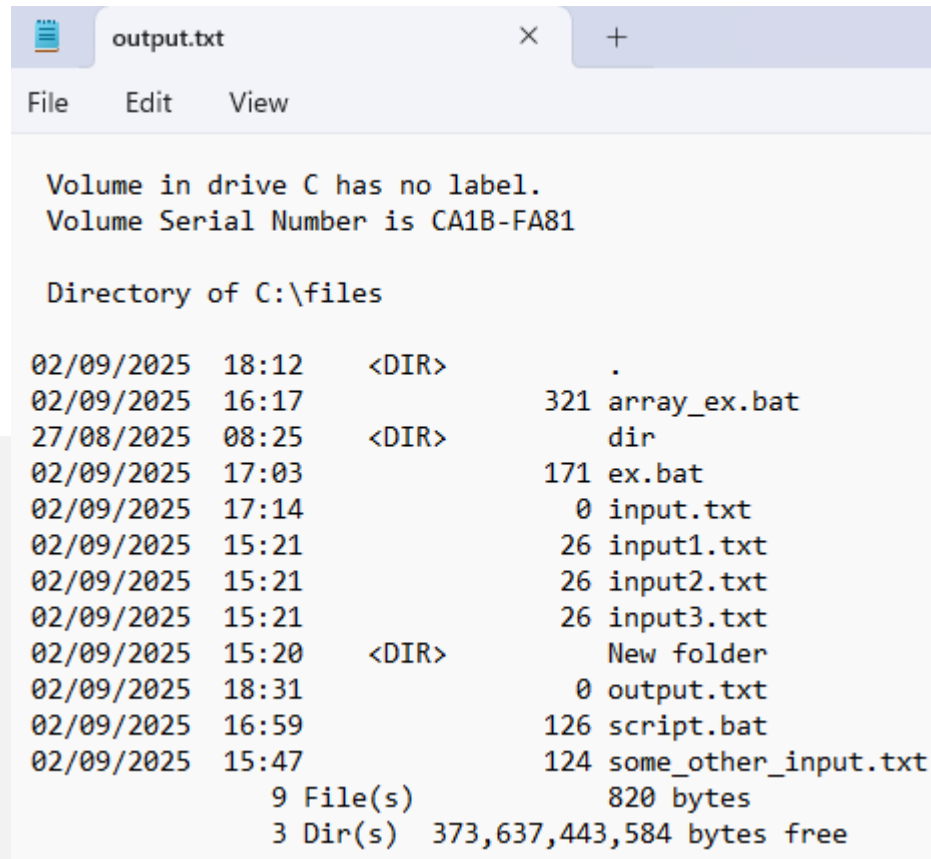
- ניתן להשתמש בProcessBuilder כדי להריץ את cmd עם פקודות מורכבות
- הפרמטר הראשון לתוכנית (הערך השני בבנאי) צריך להיות הקונ בן נמצאים
- הפרמטר הבא יהיה הפקודה שנרצה להריץ
- לדוגמא: הרצת פקודה שתרשום טקסט לקובץ, ולאחר מכן תפתח את הקובץ

```
String file = "C:\\files\\output.txt";
ProcessBuilder pb = new ProcessBuilder("cmd.exe", "/C", "echo hello world>" + file);
ProcessBuilder pbn = new ProcessBuilder("notepad.exe", file);
try {
    pb.start().waitFor();
    pbn.start();
} catch (Exception e) {
    e.printStackTrace();
}
```



הרצת תהליך

```
String file = "C:\\files\\output.txt";
ProcessBuilder pb = new ProcessBuilder("cmd.exe", "/C", "dir>" + file);
ProcessBuilder pbn = new ProcessBuilder("notepad.exe", file);
pb.directory(new File("C:\\files"));
try {
    pb.start().waitFor();
    pbn.start();
} catch (Exception e) {
    e.printStackTrace();
}
```



```
Volume in drive C has no label.
Volume Serial Number is CA1B-FA81

Directory of C:\files

02/09/2025  18:12    <DIR>          .
02/09/2025  16:17          321 array_ex.bat
27/08/2025  08:25    <DIR>          dir
02/09/2025  17:03          171 ex.bat
02/09/2025  17:14             0 input.txt
02/09/2025  15:21            26 input1.txt
02/09/2025  15:21            26 input2.txt
02/09/2025  15:21            26 input3.txt
02/09/2025  15:20    <DIR>        New folder
02/09/2025  18:31             0 output.txt
02/09/2025  16:59          126 script.bat
02/09/2025  15:47          124 some_other_input.txt
               9 File(s)              820 bytes
               3 Dir(s)  373,637,443,584 bytes free
```

- הטיפוס File מייצג קובץ או תיקייה
- לProcessBuilder שיטה directory(File) אשר מקבלת תיקייה וקובעת אותה להיות התיקייה בה התהליך ירוץ
- טיפוס זה מובנה בJAVA אך יש לבצע ייבוא מהחבילה java.io
`import java.io.File;`

הרצת תהליך

- לעיתים נרצה להשתמש בפלט שהתקבל מהתהליך בתוך התוכנה שלנו
- נכיר מספר טיפוסים שנצטרך לעשות בהם שימוש כדי לבצע זאת
- הטיפוס `InputStream` מייצג אובייקט שיודע לקבל לתוכו קלט
- הוא יקבל את הפלט של התוכנה (הפלט של התוכנה נכנס כקלט לאובייקט)
- הטיפוס `InputStreamReader` מייצג אובייקט שיודע לקרוא נתונים מ-`InputStream`
- הטיפוס `BufferedReader` מייצג אובייקט שיודע לקחת את הנתונים ש-`InputStreamReader` קרא ולהעביר אותם לנו כמחרוזת
- כל הטיפוסים הללו מובנים ב-JAVA ויש לבצע ייבוא להם מהחבילה `java.io`

```
import java.io.*;
```

אפשר
גם

```
import java.io.BufferedReader;  
import java.io.File;  
import java.io.InputStream;  
import java.io.InputStreamReader;
```

הרצת תהליך

- לעיתים נרצה להשתמש בפלט שהתקבל מהתהליך בתוך התוכנה שלנו
- ל-Procces יש את השיטה `getInputStream()` אשר מביאה לנו את `InputStream` שמכיל את הפלט שלו
- נשתמש בבנאי של `InputStreamReader` עם `InputStream` שקיבלנו
- נשתמש בבנאי של `BufferedReader` עם `InputStreamReader` שיצרנו

הגדרת הפקודה – הצגת תוכן התיקייה

```
ProcessBuilder pb = new ProcessBuilder("cmd.exe", "/C", "dir");  
pb.directory(new File("C:\\files"));
```

הגדרת התיקייה בה תרוץ הפקודה

```
try {
```

```
    Process p = pb.start();
```

הרצת התהליך והמתנה לסיומו

```
    p.waitFor();
```

```
    InputStream is = p.getInputStream();
```

קבלת ה-InputStream

```
    BufferedReader reader = new BufferedReader(new InputStreamReader(is));
```

```
    String line;
```

```
    System.out.println("the output:");
```

יצירת ה-InputStreamReader
וה-BufferedReader

```
    int i=1;
```

```
    while ((line = reader.readLine()) != null)
```

```
        System.out.println((i++)+":\t"+line);
```

קריאת הפלט שורה אחר שורה
והדפסתו

```
} catch (Exception e) {
```

```
    e.printStackTrace();
```

```
}
```

the output:

```
1:      Volume in drive C has no label.
2:      Volume Serial Number is CA1B-FA81
3:
4:      Directory of C:\files
5:
6:      02/09/2025   18:12    <DIR>          .
7:      02/09/2025   16:17                321 array_ex.bat
8:      27/08/2025   08:25    <DIR>          dir
9:      02/09/2025   17:03                171 ex.bat
10:     02/09/2025   17:14                 0 input.txt
11:     02/09/2025   15:21                 26 input1.txt
12:     02/09/2025   15:21                 26 input2.txt
13:     02/09/2025   15:21                 26 input3.txt
14:     02/09/2025   15:20    <DIR>          New folder
15:     02/09/2025   18:31                765 output.txt
16:     02/09/2025   16:59                126 script.bat
17:     02/09/2025   15:47                124 some_other_input.txt
18:                  9 File(s)                1,585 bytes
19:                  3 Dir(s)  373,740,097,536 bytes free
```

הרצת תהליך

- כפי שראינו ניתן להוסיף לפקודה גם ארגומטים, למשל אם נרצה לבצע dir בתצורה מצומצמת, נוכל לכתוב `new ProcessBuilder("cmd.exe", "/C", "dir /b")`
- ניתן להוסיף ארגומנטים גם לערכים שניתנים לבנאי `new ProcessBuilder("cmd.exe", "/C", "dir", "/b")`
- בשניהם נקבל אותה תוצאה
- אז מה ההבדל?
- בשיטה הראשונה אנו מסתמכים על התחביר של הסביבה המסויימת בה אנו עובדים
- בשיטה השנייה אנו מגדירים בצורה יותר אבסטרקטית- האובייקט יודע שזה ארגומנט ובאחריותו לעצב את הפקודה שהוא שולח למערכת בצורה נכונה
- מה עדיף?
- השיטה השנייה

the output:

```
1:      array_ex.bat
2:      dir
3:      ex.bat
4:      input.txt
5:      input1.txt
6:      input2.txt
7:      input3.txt
8:      New folder
9:      output.txt
10:     script.bat
11:     some_other_input.txt
```

הרצת תהליך

- בבנית ProcessBuilder להרצת תהליך נעדיף להגדיר באובייקט עצמו מה הארגומנטים ולא להסתמך על התחביר המסויים
- באופן כזה נקבל תוכנה שמותאמת ליותר סביבות
- לדוגמא נניח שאנו רוצים להריץ את התוכנה במערכת הפעלה בה תחביר הפקודות הוא בדיוק אותו דבר, רק שלפני כל ארגומנט צריך להוסיף !, אחרת המערכת מתעלמת ממנו
- בשיטה הראשונה `ProcessBuilder("cmd.exe", "/C", "dir /b");`
- הפקודה שתרוץ היא "dir /b" ומערכת ההפעלה תתעלם מהארגומנט
- בשיטה השניה `new ProcessBuilder("cmd.exe", "/C", "dir", "/b");`
- האובייקט יידע להתאים לתחביר והפקודה שתרוץ תהיה "dir !/b" והארגומנט ייקרא

הרצת תהליך

- גם ניתוב בפלט ניתן לקבוע דרך האובייקט

- במקום

```
ProcessBuilder pb = new ProcessBuilder("cmd.exe", "/C", "dir>C:\\files\\output.txt");
```

- נגדיר

```
ProcessBuilder pb = new ProcessBuilder("cmd.exe", "/C", "dir");  
pb.redirectOutput(new File("C:\\files\\output.txt"));
```

- בדומה לארגומנטים באופן זה נקבל הגדרה אבסטרקטית שאינה תלויה בתחביר המסויים

הרצת תהליך

- אנו בעצם קיבלנו פה טעימה של תכנות אסינכרוני – תכנות שבו במקום שיש חישוב אחד שרץ מהתחלה עד הסוף מספר חישובים שונים מבצעים את המשימה
- בדוגמא שלנו: תוכנת ה-JAVA וה-CMD
- בנושא הבא נתנסה בתכנות אסינכרוני מורכב ויותר ונכיר את היכולות שהוא מספק לנו ביחד עם האתגרים שהוא מציב מולנו